

Design Rationale

REQ 1 FIT 2099 A2

Name: Youssef Ahmed Adel Attia Hassanein

Student ID: 34073817

A. Spirit Goat Production

Design 1: In-class reproduction logic by implementing the offspring spawning directly in SpiritGoat class

Pros	Cons
Simple no extra classes needed	Violates Single Responsibility Principle (SRP) by combining movement and reproduction in one class
Direct access to the internals of the class	Violates Open/Closed Principle (OCP) because adding new reproduction triggers requires modifying SpiritGoat
	Hard to reuse or extend for the use by other animals / producers

Design 2: Have SpiritGoat implement Producible, and inject a ProduceBehaviour that queries adjacency via AdjacentCapabilityChecker

Pros	Cons
SRP: reproduction logic isolated in ProduceBehaviour.	Slightly more upfront complexity (interface + behaviour class).
OCP: other creatures can be made reproducible without touching SpiritGoat.	Extra classes to learn and maintain.
Easy modification and extension	

B. Omen Sheep Egg Production Mechanism

Design 1: Every seventh turn, OmenSheep directly creates an Egg item and places it on its tile.

Pros	Cons
Simple implementation	Mixes reproduction with movement; violates SRP.
No need for additional interfaces or behaviours.	Breaks OCP: new egg timers require editing OmenSheep.
	Difficult to reuse for other egg-laying creatures.

Design 2: Leverage the same Producible interface and ProduceBehaviour, with OmenSheep.canProduce() returning true every seven turns.

Pros	Cons
Reuses generic reproduction infrastructure.	Requires managing turn counters within behaviour.
SRP/OCP: separates egg timing from core class.	More classes to coordinate (behaviour + interface).
Uniform API for all producers.	

C. Egg Hatching Mechanism

Design 1: Hatching in actor classes

Embed hatching logic in each egg-specific class, checking ground adjacency or turn count.

Pros	Cons
Direct control over when and where hatching occurs.	Duplicated logic across different egg types; violates DRY.
Simple if only one egg type exists.	Hard to extend to new hatching conditions.

Design 2: Create an Egg base class that overrides tick(), manages a turn counter or adjacency checks, and invokes hatch() when conditions met.

Pros	Cons
Encapsulates lifecycle in one place; SRP.	Increases abstraction overhead.
Supports multiple egg types via subclassing.	Has to have good understanding of the tick system
OCP: add new eggs without modifying base engine.	

D. Egg Consumption Mechanism

Design 1: Override getAllowableActions() and eat() in concrete egg (e.g. OmenEgg) to directly boost Farmer's max HP.

Pros	Cons
SRP: logic stays with egg class.	Duplicates common eating logic if multiple consumables.
	Difficult to standardize eatable items.

Design 2: Define an Eatable interface and generic EatAction that invokes eatable.eat(actor, map).

Pros	Cons
DRY: centralized action; can apply to any Eatable.	Must ensure correct action registration.
OCP: new consumables need only implement Eatable.	

Final design:

- SpiritGoat & OmenSheep implement Producible and manage reproduction by sharing ProduceBehaviour. This cleanly separates behaviour from core class (SRP), and allows future producers (e.g., blessed plants) to reuse the same interface (OCP, DIP).
- Abstract Egg extends Item, manages its own tick()–based hatching logic (SRP, LSP). Concrete eggs only override createHatchling() and shouldHatch().
- Eatable + EatAction handle consumption: eggs implement Eatable.eat(), and any actor picks them up and eats via EatAction

SOLID principles:

- Single Responsibility: behaviours, lifecycle, and consumption each in their own classes/interfaces.
- Open/Closed: new producers, eggs, or consumables added without modifying existing code.
- Liskov Substitution: any Producible, Egg, or Eatable subclass can be used interchangeably.
- Interface Segregation: light-weight Producible, Eatable, and Curable interfaces.
- Dependency Inversion: high-level game logic depends on abstractions (Behaviour, interfaces), not concretions.